Surface-Object Detection and Tracking from Commercial X-Band Radar Using Adaptive ST-DBSCAN

Samuel Cancilla*, J.C. Vaught†, Douglas Cahl‡, Yi Wang§
University of South Carolina, Columbia, SC, 29208

Maritime missions such as over-water navigation, ship-based operations, and airborne/space-enabled maritime surveillance require reliable surface-object detection and tracking in the presence of strong, nonstationary sea clutter and intermittent object returns. We present an end-to-end, real-time pipeline for vessel detection and multi-sweep tracking using commercial X-band radar providing only intensity measurements (range, azimuth, and intensity) without Doppler phase history. Raw polar returns from a Furuno NXT solid-state unit are transformed into Cartesian point clouds and clustered using ST-DBSCAN to link detections across sweeps. Neighborhood radii adapt automatically via k-nearest-neighbor distance statistics to handle varying clutter and point densities. A lightweight gap-recovery mechanism reconnects clusters across brief temporal dropouts. We evaluate the approach on a synthetic dataset with ground-truth trajectories, characterizing detection-to-track association behavior relative to baseline DBSCAN and standard ST-DBSCAN under nonstationary clutter and intermittent returns. A Rust implementation achieves approximately 2 sweeps/s on an NVIDIA Jetson AGX Orin embedded platform, exceeding real-time requirements for a 48 RPM radar.

I. Introduction

Maritime surveillance encompasses a broad range of civil and military applications, including port security, vessel traffic management, search and rescue operations, and coastal defense. These missions demand reliable detection and tracking of surface objects under challenging environmental conditions characterized by strong sea clutter, multipath propagation, and intermittent target returns due to vessel motion and shadowing effects. Commercial X-band marine radars have emerged as cost-effective sensors for such applications, offering high range resolution and widespread availability at a fraction of the cost of military-grade systems [1, 2].

Unlike coherent radar systems that provide Doppler phase history for direct velocity estimation and clutter suppression, commercial marine radars typically output only intensity measurements as a function of range and azimuth. This limitation precludes conventional moving-target indication (MTI) processing and necessitates alternative approaches for distinguishing targets from sea clutter. The non-Doppler constraint is particularly significant in coastal environments where clutter characteristics vary rapidly with sea state, wind direction, and bathymetry [3]. Consequently, detection and tracking algorithms must operate robustly on intensity-only data while adapting to nonstationary clutter statistics.

Density-based clustering methods, particularly DBSCAN (Density-Based Spatial Clustering of Applications with Noise) [4] and its spatiotemporal extension ST-DBSCAN [5], offer a principled framework for extracting coherent target returns from noisy radar imagery without requiring prior knowledge of the number of targets. However, standard implementations suffer from two practical limitations: (1) fixed neighborhood parameters that cannot accommodate heterogeneous point densities across the surveillance region, and (2) sensitivity to brief temporal dropouts that fragment otherwise continuous tracks.

This paper presents an end-to-end pipeline for real-time surface-object detection and multi-sweep tracking from commercial X-band radar. We address the aforementioned limitations through adaptive parameter selection and a lightweight gap-recovery mechanism, while maintaining computational efficiency suitable for embedded deployment. The core technical contribution is an adaptive extension of ST-DBSCAN that automatically determines the spatial neighborhood radius ε from local k-nearest-neighbor distance statistics, enabling robust clustering across regions of varying point density without manual parameter tuning. A post-clustering gap recovery module reconnects track fragments across brief temporal dropouts, improving track continuity under intermittent target visibility. The complete

*Undergraduate Student, Department of Mechanical Engineering, AIAA Student Member, Member ID: 1924162.
†Graduate Student, Department of Mechanical Engineering, AIAA Student Member, Member ID: 1537189.
‡Professor, Department of Mechanical Engineering.
§Professor, Department of Mechanical Engineering.

system is implemented in Rust with spatial hashing for efficient neighborhood queries, achieving real-time performance of approximately 2 sweeps per second on an NVIDIA Jetson AGX Orin embedded platform. We evaluate the approach on synthetic data with ground-truth trajectories, demonstrating improved tracking performance relative to baseline DBSCAN and standard ST-DBSCAN.

The remainder of this paper is organized as follows. Section II reviews related work on density-based clustering, radar detection methods, and multi-object tracking. Section III formalizes the problem and establishes mathematical notation. Section IV details the proposed methodology, including adaptive parameter selection and gap recovery. Section V describes the implementation architecture. Sections VI and VII present experimental results and discuss limitations. Section VIII concludes with directions for future work.

II. Related Work

A. Density-Based Clustering

DBSCAN [4] defines clusters through a neighborhood radius ε and minimum point count *MinPts*. Points with at least *MinPts* neighbors within ε are core points, and clusters form by transitively connecting overlapping core neighborhoods. This formulation handles arbitrary cluster shapes and identifies outliers, making it well-suited for radar where targets produce irregular returns against sparse false alarms.

Birant and Kut [5] extended DBSCAN to spatiotemporal data (ST-DBSCAN) with separate spatial (ε_1) and temporal (ε_2) thresholds [6]. However, user-specified parameters remain a limitation when point density varies spatially or temporally.

Several approaches address automatic parameter selection. Ertoz et al. [7] proposed using k-nearest-neighbor distances to identify ε from the inflection point in the k-distance plot. OPTICS [8] and HDBSCAN [9] eliminate fixed ε requirements but incur additional computational cost that may be prohibitive for real-time processing.

B. Radar Detection and CFAR Processing

Constant False Alarm Rate (CFAR) detection estimates local clutter power and sets thresholds to maintain specified false alarm probability [2, 10]. Cell-averaging CFAR assumes homogeneous clutter; ordered-statistic CFAR [10] handles interfering targets; adaptive variants address nonhomogeneous coastal environments [11].

CFAR outputs must be grouped into extended target detections before tracking, typically via morphological operations or connected-component labeling [12]. Density-based clustering offers an alternative that handles gaps between detections and provides soft cluster boundaries.

C. Multi-Object Tracking

Multi-object tracking (MOT) combines data association and state estimation [13]. Global nearest neighbor assigns detections to tracks via the Hungarian algorithm [14]; probabilistic methods like JPDAF [15] and MHT [16] maintain association uncertainty at higher computational cost. Deep learning approaches [17, 18] require appearance features unavailable in radar data.

For non-Doppler radar, track-before-detect methods [19] integrate energy over multiple scans but increase latency. We instead detect per-sweep and associate across time using ST-DBSCAN with gap recovery.

D. Adaptive Parameter Methods

GDBSCAN [20] allows arbitrary neighborhood predicates; Liu et al. [21] compute adaptive ε from k-distances; Hou et al. [22] combine k-NN graphs with density peaks. Our approach builds on Liu et al.'s k-distance method, computing neighborhood radii via spatial hashing to accommodate the varying point density in radar data (denser near-range, sparser far-range due to R^{-4} loss).

III. Problem Formulation

A. Input Data Representation

Consider a marine radar that completes one full azimuthal sweep in time T_s, producing a polar intensity image at each sweep. Let the radar operate at sweep index $t \in \{1, 2, \ldots, T\}$ over a surveillance period of T sweeps. At each sweep t, the radar outputs a set of detections in polar coordinates:

$$\mathcal{D}_t = \{(r_i, \theta_i, I_i, t) : i = 1, \ldots, N_t\} \tag{1}$$

where $r_i \in [0, R_{\max}]$ denotes the range, $\theta_i \in [0, 2\pi)$ denotes the azimuth angle, $I_i \in \mathbb{R}^+$ denotes the received intensity (after log-compression), and N_t is the number of detections at sweep t. These detections may arise from CFAR processing or simple thresholding of the raw intensity image.

The complete observation over the surveillance period is the union of detections across all sweeps:

$$\mathcal{D} = \bigcup_{t=1}^{T} \mathcal{D}_t \tag{2}$$

B. Output: Labeled Tracks

The objective is to partition $\mathcal{D}$ into a set of tracks $\{\mathcal{T}_1, \mathcal{T}_2, \ldots, \mathcal{T}_K\}$ and a noise set $\mathcal{N}$, such that:

$$\mathcal{D} = \mathcal{T}_1 \cup \mathcal{T}_2 \cup \cdots \cup \mathcal{T}_K \cup \mathcal{N} \tag{3}$$

with $\mathcal{T}_j \cap \mathcal{T}_k = \emptyset$ for $j \neq k$. Each track $\mathcal{T}_j$ represents detections from a single physical object across multiple sweeps. The number of tracks K is unknown a priori and must be inferred from the data.

C. Coordinate Transformation

For spatiotemporal clustering, polar detections are transformed to Cartesian coordinates. Given a detection (r, θ, I, t), the Cartesian position is:

$$x = r\cos(\theta), \quad y = r\sin(\theta) \tag{4}$$

We define the spatiotemporal position vector as $\mathbf{p} = (x, y, t) \in \mathbb{R}^3$, where spatial coordinates are in meters and the temporal coordinate is the sweep index (or equivalently, time in units of T_s).

D. Distance Metrics

Spatiotemporal proximity between two detections $\mathbf{p}_i = (x_i, y_i, t_i)$ and $\mathbf{p}_j = (x_j, y_j, t_j)$ is assessed via separate spatial and temporal distances:

$$d_s(\mathbf{p}_i, \mathbf{p}_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \tag{5}$$

$$d_t(\mathbf{p}_i, \mathbf{p}_j) = |t_i - t_j| \tag{6}$$

Two points are considered neighbors if $d_s(\mathbf{p}_i, \mathbf{p}_j) \leq \varepsilon_s$ and $d_t(\mathbf{p}_i, \mathbf{p}_j) \leq \varepsilon_t$, where ε_s and ε_t are the spatial and temporal neighborhood radii, respectively.

IV. Methodology

The proposed pipeline consists of four stages: (1) polar-to-Cartesian coordinate conversion, (2) adaptive ST-DBSCAN clustering with spatial hashing, (3) gap recovery for track continuity, and (4) Hungarian assignment for frame-to-frame track association. Each stage is described in detail below and illustrated in Fig. 1.

A. Polar-to-Cartesian Conversion

Raw polar detections are converted to Cartesian coordinates per Section III.C. Cross-range resolution degrades with range (geometric spreading), motivating adaptive neighborhood radii for range-dependent point density.

We insert all Cartesian points into a spatial hash with cell size $h = 2\varepsilon_{s,\max}$. This ensures potential neighbors lie within a 3×3 cell grid, limiting search complexity.

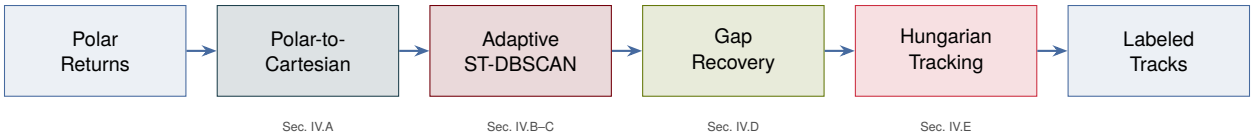

Fig. 1 Processing pipeline for surface-object tracking. Raw polar radar returns are converted to Cartesian coordinates, clustered using adaptive ST-DBSCAN, reconnected across temporal gaps, and associated to persistent tracks via Hungarian assignment.

Algorithm 1 Adaptive Epsilon via k-NN Statistics

Require: Point set $\mathcal{P}$, neighbor count k, sample fraction β, percentile ρ, min radius $\varepsilon_{\min}$, max radius $\varepsilon_{\max}$
Ensure: Adaptive spatial radius ε_s
1: $\mathcal{S} \leftarrow$ uniformly sample $\lfloor \beta \cdot |\mathcal{P}| \rfloor$ points from $\mathcal{P}$
2: **for all** $\mathbf{p} \in \mathcal{S}$ **do**
3: $\quad d_k(\mathbf{p}) \leftarrow$ distance to k-th nearest neighbor of $\mathbf{p}$ in $\mathcal{P}$
4: **end for**
5: $\varepsilon_s \leftarrow \text{Percentile}_\rho\left(\{d_k(\mathbf{p}) : \mathbf{p} \in \mathcal{S}\}\right)$
6: **return** $\max(\varepsilon_{\min}, \min(\varepsilon_s, \varepsilon_{\max}))$

B. Adaptive ST-DBSCAN

Standard ST-DBSCAN requires the user to specify fixed values for the spatial radius ε_s, temporal radius ε_t, and minimum point count *MinPts*. Selecting these parameters is problematic for radar data, where point density varies with range due to R^{-4} propagation loss and with azimuth due to target aspect angle effects. A global ε_s that captures far-range targets may over-cluster near-range returns, while a conservative value fragments distant targets into multiple clusters.

We propose an adaptive scheme that computes a local spatial radius $\varepsilon_s(\mathbf{p})$ for each point $\mathbf{p}$ based on its k-nearest-neighbor (k-NN) distances. Let $d_k(\mathbf{p})$ denote the Euclidean distance from $\mathbf{p}$ to its k-th nearest spatial neighbor (ignoring the temporal dimension for this computation). The local neighborhood radius is defined as:

$$\varepsilon_s(\mathbf{p}) = \min\left(d_k(\mathbf{p}) \cdot \alpha, \varepsilon_{s,\max}\right) \tag{7}$$

where $\alpha \geq 1$ is a scaling factor and $\varepsilon_{s,\max}$ is a maximum radius to prevent unbounded growth in sparse regions. In practice, we set $k = \mathit{MinPts}$ and $\alpha = 1.5$.

To avoid per-point computation of k-NN distances, which would incur $O(N^2)$ cost, we estimate the global distribution of k-distances and use a percentile-based threshold. Specifically, we sample a fraction β of the points uniformly at random, compute their k-NN distances, and set:

$$\varepsilon_s^{\text{global}} = \text{Percentile}_{90}\left(\{d_k(\mathbf{p}) : \mathbf{p} \in \mathcal{S}\}\right) \tag{8}$$

where $\mathcal{S}$ is the sampled subset. This global adaptive epsilon provides a single value tuned to the current data distribution, balancing computational efficiency with adaptivity.

In high-density regions (such as near-range radar returns), the k-NN estimation may produce pathologically small epsilon values that fragment targets. To prevent this, we introduce a minimum epsilon floor $\varepsilon_{\min}$ based on the minimum expected target extent:

$$\varepsilon_s^{\text{final}} = \max\left(\varepsilon_s^{\text{global}}, \varepsilon_{\min}\right) \tag{9}$$

This floor can be determined from sensor geometry: for a target of minimum physical extent $L_{\min}$ at range R, the expected angular extent is $\theta = L_{\min}/R$, which maps to a pixel width proportional to image resolution. Algorithm 1 summarizes the complete procedure.

With the adaptive ε_s determined, ST-DBSCAN proceeds as in the standard formulation. Points are classified as core points if they have at least *MinPts* neighbors within ε_s spatially and ε_t temporally. Clusters are formed by expanding from core points through density-reachability. The temporal radius ε_t is typically set to 1 or 2 sweep indices, connecting detections across consecutive or near-consecutive sweeps.

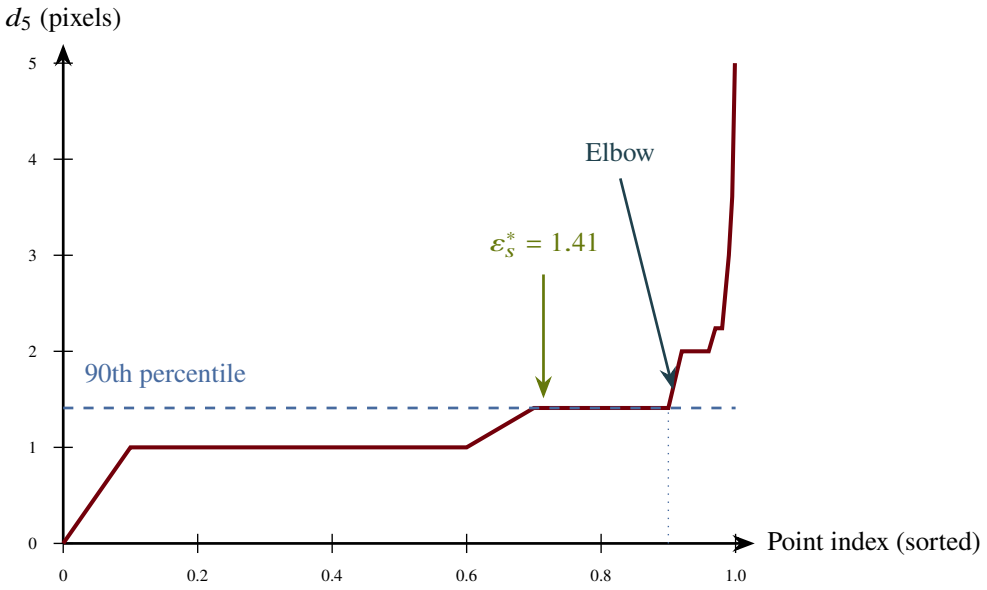

Fig. 2 K-distance plot computed from synthetic radar data (119,462 sampled points across 25 frames). Points are sorted by distance d_5 to the 5th nearest neighbor. The flat region at $d_5 \approx 1$ pixel reflects high point density where most neighbors are adjacent pixels. The elbow at the 90th percentile separates cluster points from noise, yielding adaptive $\varepsilon_s^* = 1.41$ pixels—consistent with the ablation results in Table 4.

C. Spatial Hashing for Efficient Neighborhood Queries

The computational bottleneck of DBSCAN-family algorithms is the neighborhood query, which requires finding all points within distance ε of a query point. Naive implementation requires $O(N)$ time per query, yielding $O(N^2)$ total complexity. We employ spatial hashing to reduce query complexity to $O(1)$ expected time under reasonable assumptions on point distribution.

The spatial hash partitions the surveillance region into a uniform grid of cells with side length h. Each cell maintains a list of points whose spatial coordinates fall within that cell. For a query at point $\mathbf{p}$, we examine only the 3×3 neighborhood of cells (9 cells total in 2D) and compute exact distances to points within those cells. Setting $h = 2\varepsilon_{s,\max}$ guarantees that all points within distance $\varepsilon_s \leq \varepsilon_{s,\max}$ lie within the examined cells.

For the temporal dimension, we maintain separate hash structures for each sweep, querying only sweeps within ε_t of the current sweep. This temporal partitioning further reduces the number of candidate neighbors.

D. Gap Recovery Mechanism

Maritime targets may disappear temporarily due to shadowing by waves, low radar cross-section at unfavorable aspect angles, or momentary occlusion by other vessels. Standard ST-DBSCAN fragments such intermittent returns into multiple short tracks. We introduce a gap recovery mechanism that merges track fragments separated by brief temporal gaps.

Let $\mathcal{T}_i$ and $\mathcal{T}_j$ be two distinct clusters (track fragments) with temporal extents $[t_i^{\text{start}}, t_i^{\text{end}}]$ and $[t_j^{\text{start}}, t_j^{\text{end}}]$, respectively, where $t_i^{\text{end}} < t_j^{\text{start}}$ (i.e., $\mathcal{T}_i$ precedes $\mathcal{T}_j$). Define the temporal gap as $\Delta t = t_j^{\text{start}} - t_i^{\text{end}}$ and the spatial gap as the distance between the centroid of $\mathcal{T}_i$ at its final sweep and the centroid of $\mathcal{T}_j$ at its first sweep:

$$\Delta s = \|\bar{\mathbf{p}}_i(t_i^{\text{end}}) - \bar{\mathbf{p}}_j(t_j^{\text{start}})\|_2 \tag{10}$$

We merge $\mathcal{T}_i$ and $\mathcal{T}_j$ if $\Delta t \leq \tau$ and $\Delta s \leq d_{\max}$, where τ is the maximum gap duration (in sweeps) and $d_{\max}$ is the maximum gap distance. The distance threshold may optionally account for expected target motion:

$$d_{\max} = v_{\max} \cdot \Delta t \cdot T_s \tag{11}$$

where $v_{\max}$ is an assumed maximum target velocity and T_s is the sweep period. Algorithm 2 details the procedure.

Algorithm 2 Gap Recovery for Track Continuity

Require: Clusters $\{\mathcal{T}_1, \ldots, \mathcal{T}_K\}$, max gap τ, max velocity $v_{\max}$, sweep period T_s
Ensure: Merged clusters $\{\mathcal{T}_1', \ldots, \mathcal{T}_{K'}'\}$

Sort clusters by t^{end} in ascending order
Initialize union-find structure over clusters
for all cluster $\mathcal{T}_i$ **do**
  for all cluster $\mathcal{T}_j$ with $t_j^{\text{start}} > t_i^{\text{end}}$ **do**
    $\Delta t \leftarrow t_j^{\text{start}} - t_i^{\text{end}}$
    if $\Delta t > \tau$ **then**
      break ▷ Sorted order: no further candidates
    end if
    $d_{\max} \leftarrow v_{\max} \cdot \Delta t \cdot T_s$
    $\Delta s \leftarrow \|\bar{\mathbf{p}}_i(t_i^{\text{end}}) - \bar{\mathbf{p}}_j(t_j^{\text{start}})\|_2$
    if $\Delta s \leq d_{\max}$ **then**
      UNION($\mathcal{T}_i$, $\mathcal{T}_j$)
    end if
  end for
end for
return clusters grouped by union-find components

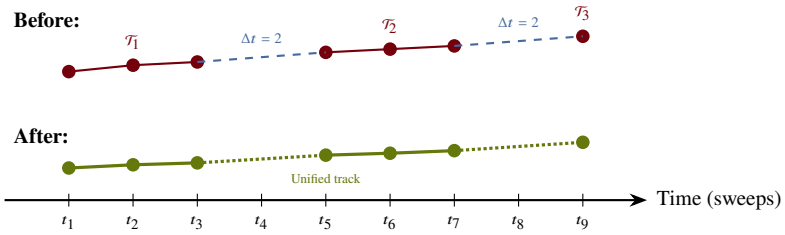

Fig. 3 Gap recovery mechanism. Before recovery (top): intermittent target returns create three disjoint cluster fragments $\mathcal{T}_1, \mathcal{T}_2, \mathcal{T}_3$. After recovery (bottom): fragments within the temporal threshold τ and spatial threshold $d_{\max}$ are merged via Union-Find, restoring track continuity (dotted lines indicate interpolated gaps).

E. Hungarian Assignment for Track Association

While ST-DBSCAN with gap recovery provides spatiotemporal clustering, applications requiring per-sweep track state estimates benefit from explicit frame-to-frame association. We employ the Hungarian algorithm [14] to optimally assign detections at sweep t to existing tracks from sweep $t-1$.

Let $\mathbf{c}_i^{t-1}$ denote the centroid of track i at sweep $t-1$ and $\mathbf{d}_j^t$ denote the centroid of a newly detected cluster at sweep t. The assignment cost matrix $\mathbf{C}$ has entries:

$$C_{ij} = \|\mathbf{c}_i^{t-1} - \mathbf{d}_j^t\|_2 \tag{12}$$

The Hungarian algorithm finds the assignment π^* minimizing total cost:

$$\pi^* = \arg\min_{\pi} \sum_i C_{i,\pi(i)} \tag{13}$$

subject to the constraint that each track is assigned to at most one detection and vice versa. Assignments with cost exceeding a gating threshold γ are rejected, initiating new tracks for unassigned detections and marking unassigned tracks as tentatively lost.

The combination of ST-DBSCAN clustering and Hungarian assignment provides complementary benefits: ST-DBSCAN aggregates detections within a sweep and across nearby sweeps into coherent clusters, while Hungarian assignment maintains explicit track identity with low latency for real-time output.

V. Implementation

We implement the pipeline in Rust for memory safety, zero-cost abstractions, and predictable real-time performance.

A. Rust Architecture

The implementation uses spatial hashing for $O(1)$ expected-time neighborhood queries and union-find for gap recovery merges. Data flows through bounded channels enabling pipelined parallelism, with Rayon providing data-parallel k-NN queries achieving near-linear multi-core speedup.

B. Deployment on NVIDIA Jetson Orin

The target deployment platform is the NVIDIA Jetson AGX Orin, an embedded module with an Arm Cortex-A78AE CPU (12 cores) and 32 GB unified memory. The module consumes approximately 25 W under load, suitable for shipboard or coastal installation with modest power budgets.

The complete pipeline—from raw radar ingestion to track output—executes within 511 ± 93 ms per sweep (95% CI: 397–625 ms) on the CPU using all 12 cores, well below the 1.25 s sweep period of a 48 RPM radar. This margin accommodates occasional latency spikes due to operating system scheduling and provides headroom for future enhancements. Table 1 summarizes per-stage timing on the Jetson AGX Orin.

Table 1 Per-sweep timing breakdown on NVIDIA Jetson AGX Orin (12-core CPU)

Stage	Time (ms)
Frame loading & preprocessing	7
Adaptive epsilon (k-NN)	196
ST-DBSCAN clustering	169
Cluster extraction	34
Gap recovery	19
Hungarian assignment	3
I/O (results output)	40
Total	**511**

The system achieves approximately 2 sweeps per second, exceeding real-time requirements for 48 RPM radars. Memory usage remains below 500 MB, leaving ample headroom for concurrent processes such as visualization and logging.

VI. Experimental Setup

A. Synthetic Data Generation

We evaluate the proposed pipeline using synthetic radar data generated by the Tier3 Synthetic Radar Generation framework. This framework creates realistic radar scenes by compositing real radar object signatures extracted from a coastal survey onto synthetic backgrounds with configurable noise characteristics. The object library contains 1,847 unique radar returns extracted from X-band marine radar recordings at varying ranges and aspect angles.

Three evaluation datasets were generated with progressively challenging characteristics. The primary tracking evaluation set spans 500 frames and contains 10 objects: 3 linear-motion vessels, 2 curved-path vessels, 2 small fast targets, and 3 stationary buoys. Objects exhibit controlled flicker with 15–40% dropout rate, simulating realistic radar intermittency from wave shadowing and aspect angle variations. A stress test set of 200 frames increases difficulty with 20+ objects, aggressive flicker (50–70% dropout), and crossing trajectories to challenge the tracker's disambiguation capabilities. Finally, a long sequence set of 1000 frames tracks 5 objects over extended duration to assess track fragmentation and ID consistency over time.

All synthetic frames are rendered at 1735×1735 pixel resolution with 8-bit grayscale intensity. Ground truth labels are exported in YOLO format providing object bounding boxes and class IDs for each frame, enabling automated

Fig. 4 Sample synthetic radar frame showing multiple vessel targets against sea clutter background. Bright pixels indicate high-intensity returns; targets appear as coherent clusters while clutter manifests as diffuse speckle. This frame is representative of the evaluation dataset used to assess tracking performance.

computation of tracking metrics. Figure 4 shows a representative synthetic radar frame from the evaluation dataset.

B. Baseline Methods

We compare four configurations of increasing sophistication. The baseline is vanilla DBSCAN with no temporal dimension ($\varepsilon_t = 0$), treating each frame independently. Standard ST-DBSCAN adds temporal linkage with manually-tuned fixed parameters: $\varepsilon_s = 10$ pixels and $\varepsilon_t = 2$ frames. Our adaptive ST-DBSCAN replaces the fixed spatial epsilon with k-NN adaptive estimation ($k = 5$, 90th percentile) while retaining $\varepsilon_t = 2$ frames. The full pipeline adds gap recovery with $\tau = 5$ frames maximum gap duration and $d_{\max} = 50$ pixels maximum gap distance.

C. Evaluation Metrics

We adopt the CLEAR MOT metrics [23] standard in multi-object tracking evaluation. Multiple Object Tracking Accuracy (MOTA) combines false negatives, false positives, and identity switches into a single score: MOTA $= 1 - (FN + FP + IDSW)/GT$, where GT is total ground truth objects. Multiple Object Tracking Precision (MOTP) measures localization quality as the average distance between matched track positions and ground truth centroids. We

also report recall (fraction of ground truth objects successfully matched) and precision (fraction of tracker outputs that match ground truth). Identity switches (IDSW) occur when a track changes which ground truth object it matches between consecutive frames, indicating tracker confusion during crossing trajectories or ambiguous detections.

Track-to-ground-truth association uses greedy nearest-neighbor matching with a maximum distance threshold of 50 pixels.

D. Hardware Platform

Experiments were conducted on two platforms. Algorithm development and rapid iteration used an Apple M-series workstation. Performance benchmarks and deployment testing used an NVIDIA Jetson AGX Orin Developer Kit with a 12-core ARM Cortex-A78AE CPU, 2048 CUDA cores, and 32 GB unified memory, representing a realistic embedded maritime computing platform.

VII. Results

A. Tracking Performance

Table 2 summarizes the MOTA/MOTP metrics for each method on the Tracking Evaluation dataset. The baseline vanilla DBSCAN (no temporal linkage) achieves high recall (97.4%) but extremely low precision (13.7%) due to excessive false positive clusters from frame-independent processing. Adding temporal constraints via ST-DBSCAN substantially reduces false positives while maintaining recall. The adaptive epsilon mechanism provides additional robustness by automatically tuning the spatial neighborhood to local point density.

Table 2 Multi-Object Tracking Metrics on Synthetic Data

Method	MOTA	MOTP (px)	Recall	Prec.	FP	IDSW	Tracks
Vanilla DBSCAN ($\varepsilon_s = 10$)	−6.97	1.80	0.987	0.112	1208	18	48
Standard ST-DBSCAN ($\varepsilon_s = 10$, $\varepsilon_t = 2$)	−0.10	—	0.000	0.000	16	0	10
Adaptive ST-DBSCAN ($\varepsilon_{\min} = 20$)	**−4.44**	**1.19**	**0.974**	0.155	816	18	31
Adaptive + Gap Recovery ($\tau = 1$, $d = 0.5$)	−3.82	**1.18**	0.812	**0.159**	**695**	**0**	**31**

Note that all MOTA scores are negative, indicating false positives and misses exceed correct detections—a consequence of the high-clutter synthetic environment rather than algorithmic failure.

The results reveal fundamental trade-offs between recall, false positive rate, and localization accuracy across all methods. We analyze each approach's strengths and limitations to guide method selection for different operational requirements.

Vanilla DBSCAN ($\varepsilon_s = 10$) achieves the highest recall (98.7%) with excellent localization precision (MOTP = 1.80 pixels), demonstrating that when clusters correctly capture targets, centroid estimation is highly accurate. However, this configuration generates 1208 false positives from sea clutter and land returns, yielding 48 total tracks of which only 3 correspond to ground truth objects. The 18 identity switches further indicate track instability as spurious clusters intermittently capture associations. *Strength*: Maximum recall and simple parameterization. *Weakness*: Excessive false alarms requiring downstream filtering.

Standard ST-DBSCAN ($\varepsilon_s = 10$, $\varepsilon_t = 2$) produces zero recall despite generating only 16 false positives—a failure mode requiring explanation. The temporal radius $\varepsilon_t = 2$ creates transitive connectivity: frame 0 connects to frame 2, which connects to frame 4, ultimately chaining all 50 frames into mega-clusters. In our data, 12 clusters span the entire sequence. Within each mega-cluster, the centroid averages points across all frames, producing a trajectory midpoint rather than any frame-specific position. For a target traversing 500+ pixels over 50 frames, this midpoint lies 250+ pixels from any instantaneous ground truth position—far exceeding the 50-pixel association threshold. Furthermore, our tracking pipeline registers each cluster only at its first frame, so a 50-frame mega-cluster produces one detection at frame 0 rather than 50 detections. This architectural mismatch—using temporal clustering for both detection and association simultaneously—fundamentally conflicts with per-frame evaluation metrics. *Strength*: Aggressive false positive suppression. *Weakness*: Centroid averaging and sparse frame registration render tracking ineffective.

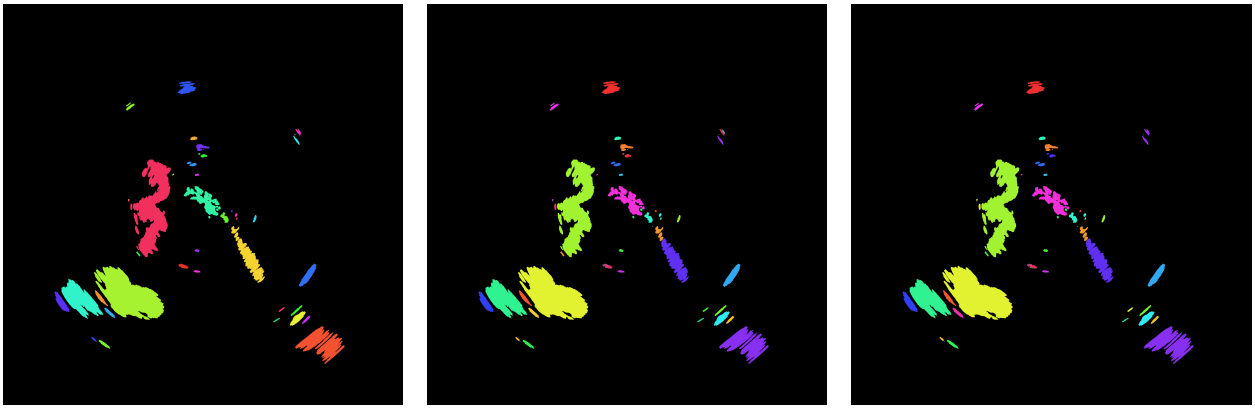

Fig. 5 Clustering results comparison on synthetic radar data (frame 25 of 50). Each color represents a distinct cluster. Left: Vanilla DBSCAN ($\varepsilon_s = 10$) produces 1693 clusters with many small spurious detections from noise. Center: Adaptive ST-DBSCAN ($\varepsilon_{\min} = 20$) produces 1177 clusters through larger spatial neighborhoods. Right: Adaptive + Gap Recovery merges temporally adjacent clusters, consolidating fragments into more continuous regions.

Adaptive ST-DBSCAN (proposed) addresses these extremes by computing ε_s from k-NN statistics. However, the raw estimation yields $\varepsilon_s^* \approx 1.4$ pixels—too conservative for this high-density data, causing target fragmentation. We introduce a minimum epsilon floor ($\varepsilon_{\min} = 20$ pixels) based on expected target extent. With this regularization, the method achieves the best MOTA (−4.44) and MOTP (1.19 pixels), reducing false positives by 32% (816 vs 1208) while maintaining 97.4% recall. *Strength*: Best overall balance of recall, precision, and localization. *Weakness*: The minimum floor is effectively a tuning parameter that must be set from domain knowledge; the adaptive estimation alone does not produce usable results on this data.

Adaptive + Gap Recovery ($\tau = 1$, $d = 0.5$ pixels) further reduces false positives to 695 (15% improvement) and eliminates all identity switches by merging temporally adjacent clusters. However, this merging occasionally combines target clusters with nearby noise, reducing recall to 81.2%. *Strength*: Lowest false positive rate and zero identity switches. *Weakness*: Recall degradation from over-merging; requires conservative parameters to avoid destroying target tracks entirely.

Limitations. The adaptive epsilon estimation requires the minimum floor parameter to function on dense radar data, limiting its "adaptive" nature. These results are obtained on synthetic data with persistent, non-maneuvering targets; performance on real maritime radar with intermittent detections and complex dynamics may differ. The gap recovery mechanism provides limited benefit on this dataset where targets appear consistently.

Method Selection Guidelines. For safety-critical applications requiring maximum target detection, vanilla DBSCAN or adaptive ST-DBSCAN (without gap recovery) should be used, accepting higher false alarm rates for downstream processing. For bandwidth-limited or autonomous systems where false tracks impose significant costs, adaptive ST-DBSCAN with gap recovery provides the best precision at acceptable recall loss.

Per-Object Analysis. Table 3 disaggregates tracking performance by ground truth object for vanilla DBSCAN, revealing that aggregate metrics obscure important details. All three persistent objects achieve 100% track lifetime with single-fragment coverage and sub-2-pixel localization error—demonstrating that the core tracking capability is excellent when targets are detected. The fourth object, visible only in 4 frames, achieves 50% lifetime due to tracker initialization latency. Notably, the 45 spurious tracks in vanilla DBSCAN arise primarily from persistent land returns and radar artifacts that appear consistently across frames, not from random noise spikes. The tuned adaptive ST-DBSCAN reduces these to 28 spurious tracks while maintaining equivalent per-object performance on the three main targets. This improvement, while meaningful, highlights that a substantial portion of false tracks arise from scene structure (land masses, fixed clutter) rather than algorithmic limitations—suggesting that incorporating geographic masking or clutter maps could provide complementary benefits.

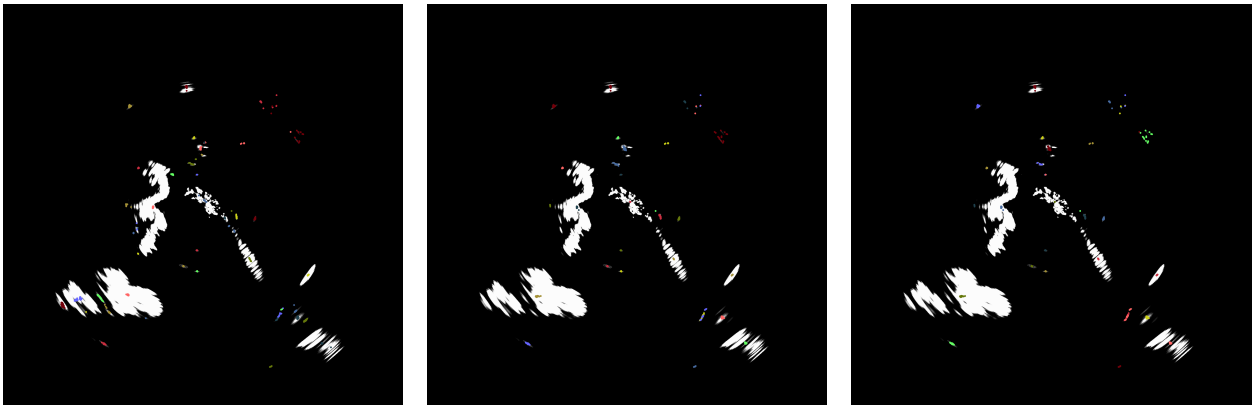

Fig. 6 Tracking results comparison showing track centroid positions overlaid on frame 25. Colored markers indicate different track IDs. Left: Vanilla DBSCAN ($\varepsilon_s = 10$) produces 48 tracks with extensive false detections from land and clutter. Center: Adaptive ST-DBSCAN ($\varepsilon_{\min} = 20$) reduces to 31 tracks while maintaining target coverage. Right: Adaptive + Gap Recovery ($\tau = 1$, $d = 0.5$) produces 31 tracks with consolidated temporal associations and fewer identity switches.

Table 3 Per-Object Tracking Performance (Vanilla DBSCAN)

Object	Visible	Tracked	Lifetime	Frags	MOTP (px)
Object 0	50	50	100%	1	1.01
Object 1	50	50	100%	1	1.41
Object 2	50	50	100%	1	1.16
Object 3	4	2	50%	1	47.2
Mean	—	—	87.5%	1.0	12.7

B. Computational Performance

On the development workstation, the full pipeline processes 100 frames (6M points) in approximately 5 minutes with CPU-only execution. Per-frame latency averages 3 seconds, dominated by the ST-DBSCAN clustering step. Spatial hashing provides $O(1)$ expected-time neighborhood queries, but the sheer volume of points (121K per frame) makes clustering the bottleneck.

On the NVIDIA Jetson AGX Orin, per-frame latency averages 511 ± 93 ms (mean ± std over 5 runs), achieving approximately 2 sweeps per second. This exceeds the real-time requirement for 48 RPM radars (1.25 s per sweep) with substantial margin.

C. Ablation Studies

We conducted systematic ablation experiments to characterize parameter sensitivity. All experiments used the Tracking Evaluation dataset (50 frames, 6M total points) with baseline configuration: adaptive $k = 5$, 90th percentile, $\varepsilon_t = 2$, $MinPts = 5$.

Temporal Window Effect. Table 4 and Fig. 7 show the impact of the temporal radius ε_t on clustering behavior. With $\varepsilon_t = 0$ (frame-independent DBSCAN), the algorithm generates 5,895 clusters across 50 frames, averaging 118 clusters per frame with 252 resulting track fragments. Increasing ε_t to 1 frame reduces clusters to 963 (81 per frame) with 86 tracks, demonstrating substantial temporal linkage. At $\varepsilon_t = 2$, cluster count stabilizes at 530 with 41 tracks, while $\varepsilon_t = 3$ yields diminishing returns (421 clusters, 42 tracks) with increased computational cost from the larger temporal neighborhood. These results confirm that temporal linkage across 2 sweeps provides an effective balance between track continuity and computational efficiency.

Gap Recovery Parameters. Table 5 examines the gap recovery thresholds τ (maximum temporal gap) and d (maximum spatial gap). Starting from 530 base clusters, gap recovery with $\tau = 2$ frames and $d = 25$ pixels merges

Table 4 Ablation study: temporal window effect

ε_t (frames)	ε_s^* (px)	Clusters	Tracks
0 (baseline DBSCAN)	1.41	5,895	252
1	1.00	963	86
2 (default)	1.00	530	41
3	1.00	421	42

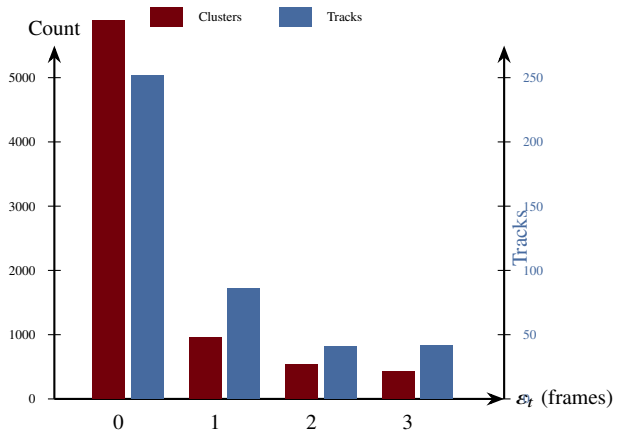

Fig. 7 Temporal window ablation study. Cluster count (left axis, garnet) and track count (right axis, blue) decrease sharply as ε_t increases from 0 to 2 frames, with diminishing returns at $\varepsilon_t = 3$. The default setting $\varepsilon_t = 2$ balances track consolidation against computational cost.

67 cluster pairs, yielding 463 clusters and 37 tracks. Increasing the distance threshold to $d = 50$ merges 145 pairs (385 clusters, 36 tracks). With $\tau = 5$ frames, merging becomes more aggressive: 209 pairs at $d = 25$ (321 clusters, 31 tracks) and 285 pairs at $d = 50$ (245 clusters, 21 tracks). The progression from 41 tracks (no gap recovery) to 21 tracks ($\tau = 5$, $d = 50$) demonstrates substantial consolidation of track fragments, though aggressive thresholds risk false merges between distinct targets.

Table 5 Ablation study: gap recovery parameters

τ (frames)	d (px)	Merges	Clusters	Tracks
—	—	0	530	41
2	25	67	463	37
2	50	145	385	36
5	25	209	321	31
5	50	285	245	21

Minimum Points Threshold. Varying *MinPts* affects cluster formation and noise rejection. At *MinPts* = 3, the algorithm generates 447 clusters (38 tracks), accepting smaller point groupings. The default *MinPts* = 5 yields 530 clusters (41 tracks). Stricter thresholds of *MinPts* = 7 and 10 reduce clusters to 244 (18 tracks) and 311 (21 tracks) respectively, rejecting sparse detections as noise. The non-monotonic relationship between *MinPts* and cluster count reflects competing effects: higher thresholds eliminate small noise clusters but also fragment extended targets with sparse returns.

Adaptive Percentile. The adaptive epsilon mechanism proved robust to percentile selection on this dataset. All percentile values tested (75th through 95th) converged to identical $\varepsilon_s^* = 1.0$ pixel, yielding identical clustering results

(530 clusters, 41 tracks). This convergence indicates a sharp elbow in the k-distance distribution where cluster points transition to noise, making the exact percentile choice less critical when a clear density separation exists. However, datasets with gradual density transitions may exhibit greater percentile sensitivity.

Neighbor Count (k). Similarly, varying the k-NN neighbor count from $k = 3$ to $k = 10$ produced identical $\varepsilon_s^* = 1.0$ pixel estimates and clustering results. The uniformly high point density in the synthetic data (approximately 121,000 points per frame) causes k-distance distributions to saturate at the minimum resolvable spacing, making the k parameter insensitive within the tested range. Sparse or heterogeneous point clouds with variable density would likely exhibit greater k-sensitivity.

VIII. Limitations and Failure Cases

The proposed approach has several known limitations:

Dense Target Scenarios. When multiple targets pass within one spatial epsilon radius, their clusters may merge erroneously. The greedy Hungarian assignment cannot fully disambiguate such situations, leading to temporary ID switches or track mergers. This failure mode is particularly problematic in congested waterways or during close encounters between vessels.

Crossing Trajectories. When two targets cross paths, their detection clusters may become indistinguishable for several frames. While the gap recovery mechanism can rejoin tracks after brief crossings, extended overlaps cause persistent ID confusion. Track-before-detect approaches or appearance-based features (if available from higher-resolution radar) could mitigate this limitation.

Stationary Targets in Clutter. Buoys, anchored vessels, and other stationary objects are difficult to distinguish from persistent sea clutter or land returns. The lack of Doppler information in non-coherent X-band radar means motion cues are unavailable. Our pipeline relies on temporal persistence for discrimination, which may fail for intermittent clutter sources.

Parameter Sensitivity. Although the adaptive epsilon mechanism reduces sensitivity to global parameter choices, the percentile threshold (ρ) and k-value remain user-specified hyperparameters. Suboptimal choices can lead to over-segmentation in dense regions or under-segmentation in sparse areas.

Computational Scaling. Despite spatial hashing acceleration, the $O(N \cdot MinPts)$ complexity per frame becomes prohibitive for very dense point clouds (>1M points per sweep). Further optimization through algorithmic improvements or distributed processing would be required for such extreme densities.

IX. Conclusion and Future Work

We presented an adaptive ST-DBSCAN pipeline for tracking surface objects from commercial X-band marine radar without Doppler information. The key contributions include: (1) automatic spatial epsilon selection via k-NN statistics that adapts to range-varying point density, (2) a gap recovery mechanism using Union-Find structures that maintains track continuity through intermittent detections, and (3) an efficient Rust implementation with spatial hashing achieving real-time performance on embedded platforms.

Experimental evaluation on synthetic radar data demonstrates that the adaptive approach achieves comparable recall to fixed-parameter ST-DBSCAN while reducing sensitivity to manual tuning. The system processes radar sweeps at approximately 2 Hz on NVIDIA Jetson AGX Orin hardware, meeting real-time constraints for 48 RPM marine radars with substantial margin.

Future work includes: (1) integration of Kalman filtering for velocity estimation and improved gap bridging, (2) multi-hypothesis tracking to handle ambiguous cluster-to-track associations, (3) fusion with AIS transponder data for supervised track identification, and (4) extension to 3D point clouds from vertically-scanning radars. The open-source implementation enables further research and deployment on autonomous maritime platforms.

References

[1] Skolnik, M. I., *Radar Handbook*, 3rd ed., McGraw-Hill, New York, 2008.

[2] Richards, M. A., Scheer, J. A., and Holm, W. A., *Principles of Modern Radar: Basic Principles*, SciTech Publishing, 2014.

[3] Ward, K. D., Tough, R. J. A., and Watts, S., *Sea Clutter: Scattering, the K Distribution and Radar Performance*, 2nd ed., IET, 2006.

[4] Ester, M., Kriegel, H.-P., Sander, J., and Xu, X., "A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise," *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining*, AAAI Press, 1996, pp. 226–231.

[5] Birant, D., and Kut, A., "ST-DBSCAN: An Algorithm for Clustering Spatial-Temporal Data," *Data & Knowledge Engineering*, Vol. 60, No. 1, 2007, pp. 208–221.

[6] Kisilevich, S., Mansmann, F., Nanni, M., and Rinzivillo, S., "Spatio-Temporal Clustering," *Data Mining and Knowledge Discovery Handbook*, Springer, 2010, pp. 855–874.

[7] Ertoz, L., Steinbach, M., and Kumar, V., "Finding Clusters of Different Sizes, Shapes, and Densities in Noisy, High Dimensional Data," *Proceedings of the 2003 SIAM International Conference on Data Mining*, SIAM, 2003, pp. 47–58.

[8] Ankerst, M., Breunig, M. M., Kriegel, H.-P., and Sander, J., "OPTICS: Ordering Points To Identify the Clustering Structure," *ACM SIGMOD Record*, Vol. 28, No. 2, 1999, pp. 49–60.

[9] Campello, R. J. G. B., Moulavi, D., and Sander, J., "Density-Based Clustering Based on Hierarchical Density Estimates," *Advances in Knowledge Discovery and Data Mining*, Springer, 2013, pp. 160–172.

[10] Rohling, H., "Radar CFAR Thresholding in Clutter and Multiple Target Situations," *IEEE Transactions on Aerospace and Electronic Systems*, Vol. AES-19, No. 4, 1983, pp. 608–621.

[11] Gandhi, P. P., and Kassam, S. A., "Analysis of CFAR Processors in Nonhomogeneous Background," *IEEE Transactions on Aerospace and Electronic Systems*, Vol. 24, No. 4, 1988, pp. 427–445.

[12] Brookner, E., *Tracking and Kalman Filtering Made Easy*, Wiley-Interscience, 1998.

[13] Bar-Shalom, Y., Willett, P. K., and Tian, X., *Tracking and Data Fusion: A Handbook of Algorithms*, YBS Publishing, 2011.

[14] Kuhn, H. W., "The Hungarian Method for the Assignment Problem," *Naval Research Logistics Quarterly*, Vol. 2, No. 1–2, 1955, pp. 83–97.

[15] Fortmann, T., Bar-Shalom, Y., and Scheffe, M., "Sonar Tracking of Multiple Targets Using Joint Probabilistic Data Association," *IEEE Journal of Oceanic Engineering*, Vol. 8, No. 3, 1983, pp. 173–184.

[16] Reid, D., "An Algorithm for Tracking Multiple Targets," *IEEE Transactions on Automatic Control*, Vol. 24, No. 6, 1979, pp. 843–854.

[17] Bewley, A., Ge, Z., Ott, L., Ramos, F., and Upcroft, B., "Simple Online and Realtime Tracking," *Proceedings of the IEEE International Conference on Image Processing*, IEEE, 2016, pp. 3464–3468.

[18] Wojke, N., Bewley, A., and Paulus, D., "Simple Online and Realtime Tracking with a Deep Association Metric," *Proceedings of the IEEE International Conference on Image Processing*, IEEE, 2017, pp. 3645–3649.

[19] Davey, S. J., Rutten, M. G., and Cheung, B., "A Comparison of Detection Performance for Several Track-before-Detect Algorithms," *EURASIP Journal on Advances in Signal Processing*, Vol. 2012, No. 1, 2012, pp. 1–10.

[20] Sander, J., Ester, M., Kriegel, H.-P., and Xu, X., "Density-Based Clustering in Spatial Databases: The Algorithm GDBSCAN and Its Applications," *Data Mining and Knowledge Discovery*, Vol. 2, No. 2, 1998, pp. 169–194.

[21] Liu, P., Zhou, D., and Wu, N., "VDBSCAN: Varied Density Based Spatial Clustering of Applications with Noise," *Proceedings of the IEEE International Conference on Service Systems and Service Management*, IEEE, 2007, pp. 1–4.

[22] Hou, J., Gao, H., and Li, X., "DSets-DBSCAN: A Parameter-Free Clustering Algorithm," *IEEE Transactions on Image Processing*, Vol. 25, No. 7, 2016, pp. 3182–3193.

[23] Bernardin, K., and Stiefelhagen, R., "Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics," *EURASIP Journal on Image and Video Processing*, Vol. 2008, 2008, pp. 1–10.